

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»

Звіт з лабораторної роботи №1

З дисципліни "Моделювання та аналітика даних"

Дослідження даних з NumPy, Pandas та scikit-learn

Виконала:
студентка гр. ПЗм-25-2
Тимощенко Владислава Русланівна

Перевірів: асистент кафедри ПІ
Дашенков Дмитро Сергійович

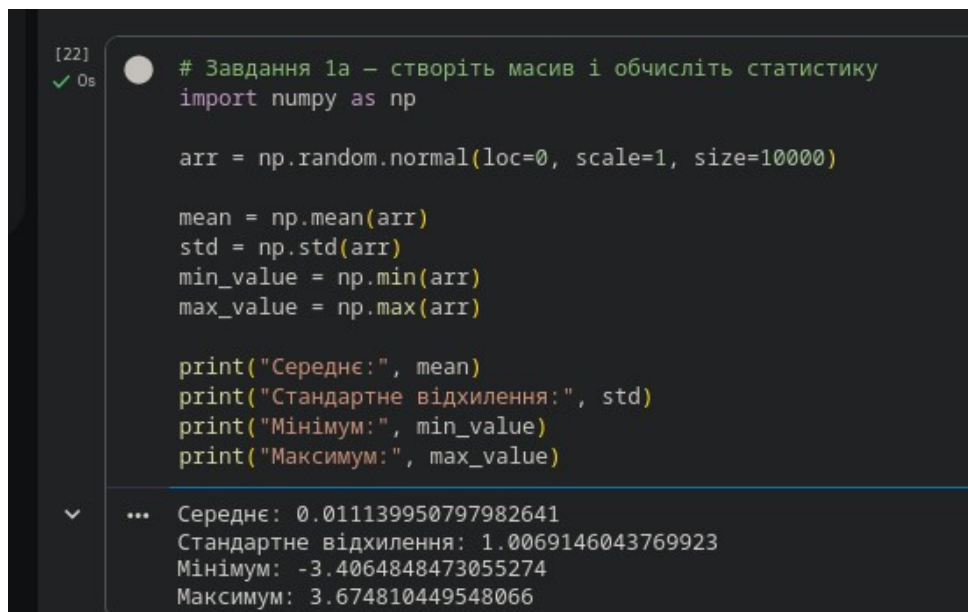
Харків 2026

1 Мета: ознайомитися з основними можливостями бібліотек NumPy, Pandas та Scikit-learn для аналізу даних, їх візуалізації та побудови моделей лінійної регресії.

2 Хід роботи

2.1 Розминка з NumPy

а) Створіть масив NumPy з 10 000 значень, що підпорядковані нормальному розподілу $N(0, 1)$. Обчисліть середнє, стандартне відхилення, мінімум і максимум за допомогою функцій NumPy — без Python-циклів.



```
[22] ✓ Os # Завдання 1a – створіть масив і обчисліть статистику
import numpy as np

arr = np.random.normal(loc=0, scale=1, size=10000)

mean = np.mean(arr)
std = np.std(arr)
min_value = np.min(arr)
max_value = np.max(arr)

print("Середнє:", mean)
print("Стандартне відхилення:", std)
print("Мінімум:", min_value)
print("Максимум:", max_value)
```

... Середнє: 0.011139950797982641
Стандартне відхилення: 1.0069146043769923
Мінімум: -3.4064848473055274
Максимум: 3.674810449548066

Створюється масив із 10 000 випадкових чисел, що мають нормальний розподіл з параметрами ($N(0,1)$). Далі обчислюється середнє значення масиву за допомогою функції `np.mean()`. Стандартне відхилення обчислюється функцією `np.std()`. Мінімальне та максимальне значення масиву визначаються за допомогою функцій `np.min()` і `np.max()`. Отримані результати виводяться на екран.

б) Підрахуйте, скільки значень виходять за межі ± 2 стандартних відхилення, використовуючи логічну індексацію. Яка це відсоткова частка? Порівняйте з теоретичним значенням $\approx 4,55\%$.

```
[24] # Завдання 16 – підрахуйте викиди за допомогою логічної індексації
✓ Os outliers = arr[np.abs(arr - mean) > 2 * std]

count_outliers = len(outliers)
percentage = count_outliers / len(arr) * 100

theoretical_percentage = 4.55
difference = abs(percentage - theoretical_percentage)

print("Кількість значень за межами  $\pm 2\sigma$ :", count_outliers)
print("Відсоткова частка:", percentage, "%")
print("Теоретичне значення:", theoretical_percentage, "%")
print("Відхилення від теоретичного значення:", difference, "%")

... Кількість значень за межами  $\pm 2\sigma$ : 464
Відсоткова частка: 4.64 %
Теоретичне значення: 4.55 %
Відхилення від теоретичного значення: 0.089999999999999986 %
```

За допомогою логічної індексації відбираються значення масиву, які знаходяться за межами двох стандартних відхилень від середнього значення. Далі підраховується кількість таких значень та обчислюється їхня відсоткова частка від загальної кількості елементів масиву. Отриманий результат порівнюється з теоретичним значенням для нормального розподілу ($\approx 4,55\%$), після чого обчислюється абсолютне відхилення між практичним і теоретичним результатами.

2.2 Завантаження та огляд набору даних

Завантажимо `kc_house_data.csv` у Pandas DataFrame та дамо відповідь на наступні питання, написавши код.

Скільки рядків і стовпців у наборі даних?

```
[27] # 2-1) Скільки рядків і стовпців?
✓ Os rows, columns = sales.shape

print("Кількість рядків:", rows)
print("Кількість стовпців:", columns)

... Кількість рядків: 21613
Кількість стовпців: 21
```

Визначається розмір набору даних за допомогою атрибута `shape`. Отримуються кількість рядків і кількість стовпців.

Які мінімум, максимум, середнє та медіана цін на будинки?

```
[28] ✓ Os # 2-ii) Мінімум, максимум, середнє та медіана ціни

min_price = sales["price"].min()
max_price = sales["price"].max()
mean_price = sales["price"].mean()
median_price = sales["price"].median()

print("Мінімальна ціна:", min_price)
print("Максимальна ціна:", max_price)
print("Середня ціна:", mean_price)
print("Медіана ціни:", median_price)

... Мінімальна ціна: 75000.0
Максимальна ціна: 7700000.0
Середня ціна: 540088.1417665294
Медіана ціни: 450000.0
```

Для стовпця `price` обчислюються мінімальне, максимальне та середнє значення, а також медіана за допомогою відповідних функцій Pandas.

Скільки унікальних поштових індексів є у наборі даних? Який поштовий індекс має найбільше оголошень?

```
[29] ✓ Os # 2-iii) Кількість унікальних поштових індексів; індекс з найбільшою кількістю оголошень

unique_zipcodes = sales["zipcode"].nunique()
most_common_zipcode = sales["zipcode"].value_counts().idxmax()

print("Кількість унікальних поштових індексів:", unique_zipcodes)
print("Поштовий індекс з найбільшою кількістю оголошень:", most_common_zipcode)

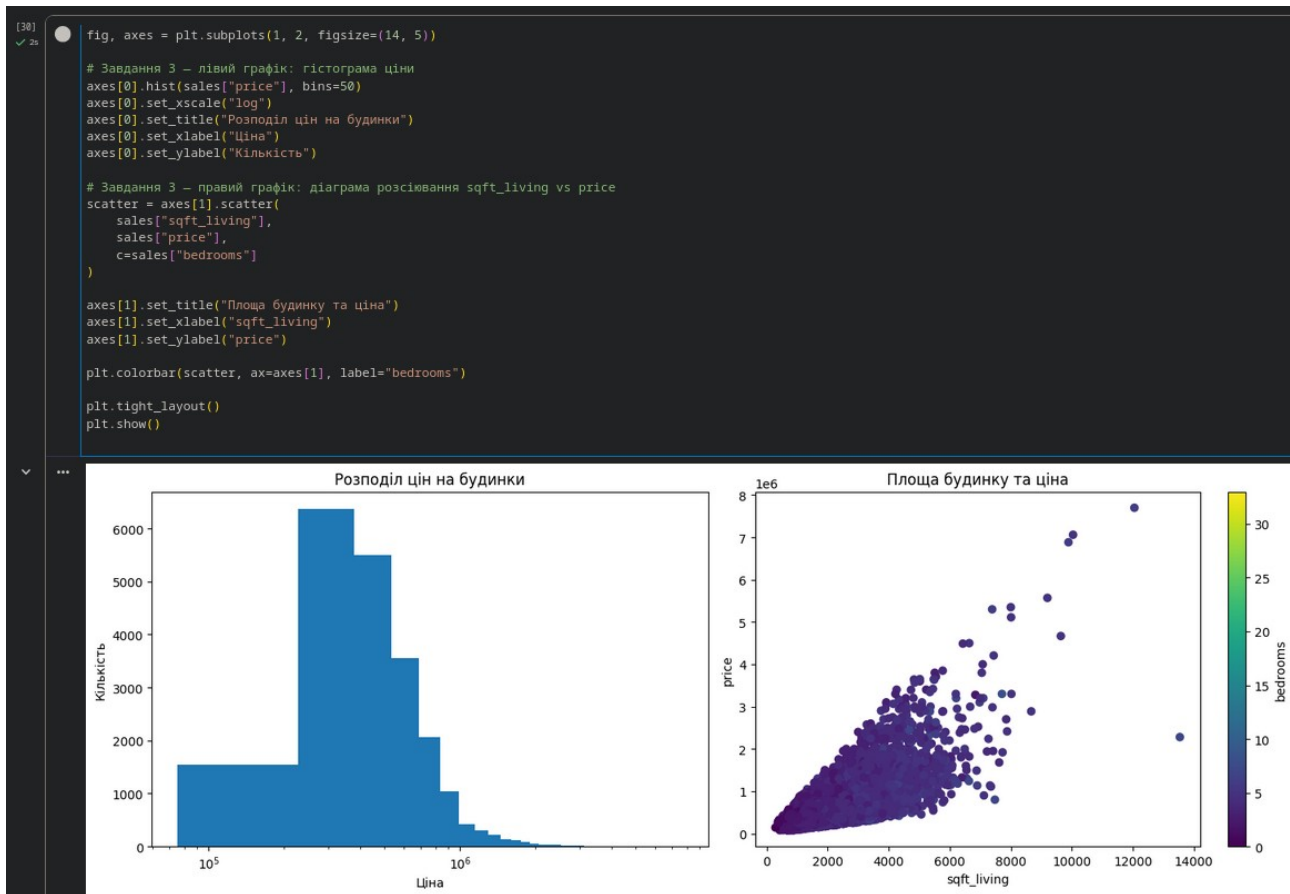
... Кількість унікальних поштових індексів: 70
Поштовий індекс з найбільшою кількістю оголошень: 98103
```

Обчислюється кількість унікальних поштових індексів у стовпці `zipcode`. Далі визначається поштовий індекс, який зустрічається найчастіше, тобто має найбільшу кількість оголошень.

2.3 Візуалізація даних

Створмо два графіки поруч:

- гістограма `price` (50 інтервалів, логарифмічна шкала по осі X).
- діаграма розсіювання `sqft_living` vs `price`, де кольором точок позначено `bedrooms`. Додайте підписи осей і `colorbar`.



Створюється фігура з двома графіками, розташованими поруч. Для лівого графіка будується гістограма цін на будинки `price` з 50 інтервалами, після чого для осі `X` встановлюється логарифмічна шкала. Для правого графіка створюється діаграма розсіювання, де по осі `X` відкладається житлова площа будинку `sqft_living`, а по осі `Y` — його ціна `price`. Колір кожної точки визначається кількістю спалень `bedrooms`, а для інтерпретації кольорів додається шкала `colorbar`. Для обох графіків задаються назви та підписи осей, після чого графіки відображаються на екрані.

2.4 Навчання та оцінка регресійної моделі

Розіб'ємо дані 80/20 (`random_state=42`). Навчимо `LinearRegression` з `scikit-learn` для передбачення `price` за єдиною ознакою `sqft_living`.

```
[32] ✓ Os # Завдання 4 — навчіть модель, вкажіть ваги, обчисліть MSE/RMSE, передбачте ціну для 2500 кв. фут.

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

X_train = train_data[["sqft_living"]]
y_train = train_data["price"]

X_test = test_data[["sqft_living"]]
y_test = test_data["price"]

model = LinearRegression()
model.fit(X_train, y_train)

intercept = model.intercept_
coefficient = model.coef_[0]

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

predicted_price = model.predict([[2500]])[0]

print("Вільний член (intercept):", intercept)
print("Коефіцієнт для sqft_living:", coefficient)
print("MSE:", mse)
print("RMSE:", rmse)
print("Передбачувана ціна для 2500 кв. футів:", predicted_price)

... Вільний член (intercept): -41999.18603993731
Коефіцієнт для sqft_living: 279.55477931526383
MSE: 76484977061.77612
RMSE: 276559.1746114674
Передбачувана ціна для 2500 кв. футів: 656887.7622482223
```

Дані поділяються на навчальну та тестову вибірки у співвідношенні 80/20. Як ознака використовується `sqft_living`, а цільовою змінною є `price`. Створюється та навчається модель лінійної регресії `LinearRegression`. Після навчання визначаються вільний член моделі (`intercept`) і коефіцієнт при ознаці `sqft_living`. Далі модель використовується для прогнозування цін на тестовій вибірці, після чого обчислюються показники якості MSE та RMSE. Наприкінці виконується прогноз ціни будинку з житловою площею 2500 квадратних футів.

2.5 Порівняння двох моделей

Навчимо другу `LinearRegression` з ознаками: `sqft_living`, `bedrooms`, `bathrooms`, `yr_built`. Порівняймо RMSE на тестовій вибірці між Моделлю 1 та Моделлю 2.

```
[41] ✓ Os # Завдання 5 – модель з чотирма ознаками та порівняння RMSE

features = ["sqft_living", "bedrooms", "bathrooms", "yr_built"]

X_train = train_data[features]
X_test = test_data[features]

y_train = train_data["price"]
y_test = test_data["price"]

model2 = LinearRegression()
model2.fit(X_train, y_train)

y_pred2 = model2.predict(X_test)

mse2 = mean_squared_error(y_test, y_pred2)
rmse2 = np.sqrt(mse2)

print("RMSE Моделі 1:", rmse)
print("RMSE Моделі 2:", rmse2)

if rmse2 < rmse:
    print("Модель 2 має нижче RMSE.")
else:
    print("Модель 1 має нижче RMSE.")

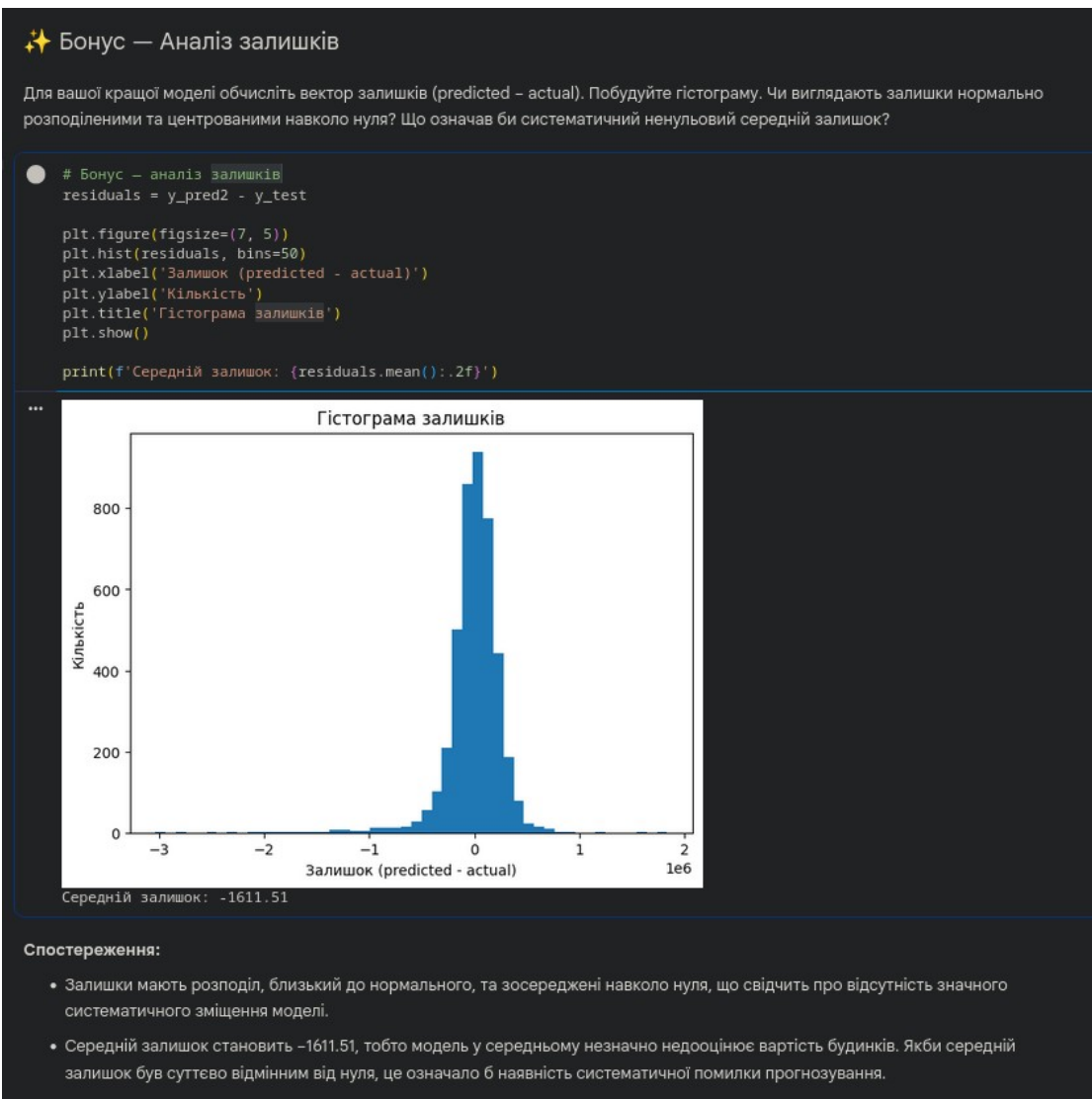
... RMSE Моделі 1: 276559.1746114674
    RMSE Моделі 2: 262985.9758719034
    Модель 2 має нижче RMSE.
```

Створюється друга модель лінійної регресії, яка використовує чотири ознаки: `sqft_living`, `bedrooms`, `bathrooms` та `yr_built`. Модель навчається на навчальній вибірці та виконує прогнозування для тестової вибірки. Після цього обчислюється показник RMSE і порівнюється з RMSE першої моделі, яка використовувала лише ознаку `sqft_living`.

Нижче значення RMSE має Модель 2, оскільки вона використовує більше ознак, які містять додаткову інформацію про будинок. Завдяки цьому модель може точніше описати залежність між характеристиками будинку та його ціною, що зменшує похибку прогнозування.

2.6 Бонус — Аналіз залишків

Для кращої моделі обчислимо вектор залишків ($\text{predicted} - \text{actual}$) та побудуємо гістограму.



Спочатку за допомогою методу `predict()` обчислюються прогнозовані значення цін для тестової вибірки. Далі формується вектор залишків як різниця між прогнозованими та реальними значеннями ($\text{predicted} - \text{actual}$). Для аналізу розподілу залишків будується гістограма з 50 інтервалами. Наприкінці обчислюється середнє значення залишків, яке дозволяє оцінити наявність систематичного зміщення моделі відносно реальних даних.

3. Висновки: в ході виконання лаборанторної роботи було виконано аналіз набору даних про продаж будинків за допомогою бібліотек NumPy та Pandas, а також побудовано візуалізації для дослідження залежностей між ознаками. Було навчено дві моделі лінійної регресії та оцінено їхню якість за допомогою показника RMSE. Встановлено, що використання більшої кількості інформативних ознак дозволяє підвищити точність прогнозування вартості будинків.

4. Покликання

Репозиторій GitHub: [покликання](#)

Відео на YouTube: [покликання](#)